

Streams, Friends & the Many Ways to Build an Object

Review worksheet — `operator<<`, `friend`, overloaded constructors, initializer lists

array[1] Find the Error

2 of 2

CASE C

COMPILES? Y / N

```
class Circle {
    const double PI;
    double r;
public:
    Circle(double radius) {
        PI = 3.14159;
        r = radius;
    }
};
```

Name the compile error and the exact line. Rewrite the constructor so it is correct.

CASE D

COMPILES? Y / N

```
class Fraction {
    int num, den;
public:
    Fraction(int n) : num(n), den(1) {}
    Fraction(int n, int d = 1)
        : num(n), den(d) {}
};

Fraction b(3);    // rejected
```

`Fraction b(3);` will not compile. Why? Give two different one-line fixes (one of them a single constructor with default arguments).

CASE E

COMPILES? Y / N

```
class Point {
    int x, y;
public:
    Point(int x, int y) { x = x; y = y; }
    friend ostream& operator<<(ostream& os, const Point& p);
};
ostream& operator<<(ostream& os, const Point& p) {
    return os << "(" << p.x << ", " << p.y << ")";
}
int main() { Point p(3, 4); cout << p; }    // prints garbage
```

The `friend` overload is fine, yet this prints garbage. What is wrong inside the constructor, and how does a member initializer list fix it in one stroke?

array[2] Short Answer

full sentences

- Explain why `operator<<` for a user-defined type is almost always a non-member function. Refer to what the left operand of `cout << obj` actually is.

- Why does the overload return `ostream&` and not `void`? Give one line of code that would fail to compile if it returned `void`, and give one reason making `operator<<` a `friend` does not really weaken encapsulation.

- Members are initialized in *declaration* order, not list order. In `class Buffer { int size; int* data; Buffer(int n) : data(new int[size]), size(n) {} };` — what goes wrong, and how do you fix it?

- Give two distinct reasons to prefer a member initializer list over plain assignment in the constructor body.
